# Δομές Δεδομένων

2η Εργασία

**Μέρος Α**

Η εκτέλεση του προγράμματος, γίνεται από την γραμμή εντολών με την μορφή:

**java Influenza_k <k> <input_file>**

όπου k το πλήθος των πόλεων με την μικρότερη πυκνότητα κρουσμάτων και input_file το όνομα του txt αρχείου. Όταν το αρχείο δεν είναι στο ίδιο φάκελο με το πρόγραμμα, τότε πρέπει να δώσουμε το path. Ως παράδειγμα, αν ο κώδικας είναι στο φάκελο src, και το αρχείο εισόδου έχει όνομα inputfile.txt και είναι στον φάκελο data, που βρίσκεται μέσα στον φάκελο src, τότε ως όνομα αρχείου πρέπει να δώσουμε data/inputfile.txt.

Τα δεδομένα σε κάθε γραμμή του αρχείου txt είναι χωρισμένα με space και αυτός ο χαρακτήρας χρησιμοποιείται για να διαχωρίσουμε τις τιμές που διαβάζουμε από το αρχείο.

Για την ταξινόμηση των πόλεων στην κλάση Influenza_k.java επιλέξαμε τον αλγόριθμο **Quicksort**. Ο αλγόριθμος Quicksort βασίζεται στον διαχωρισμό (partition) του πίνακα σε υποπίνακες.

Συνοπτικά, ο Quicksort βασίζεται στην ιδέα "διαίρει και βασίλευε" και λειτουργεί ως εξής:

1. Επιλογή Pivot: Επιλέγουμε ένα στοιχείο από τον πίνακα, το οποίο ονομάζεται Pivot. Στην περίπτωσή μας, το Pivot είναι το **τελευταίο** στοιχείο του υποπίνακα που πρέπει να ταξινομηθεί.
2. Διαχωρισμός: Ο πίνακας διαχωρίζεται (partition) σε δύο υποπίνακες - έναν υποπίνακα με στοιχεία μικρότερα από το Pivot και έναν υποπίνακα με στοιχεία μεγαλύτερα από το Pivot.
3. Αναδρομική Εφαρμογή: Εφαρμόζουμε τον αλγόριθμο Quicksort αναδρομικά στους υποπίνακες που δημιουργούνται στο βήμα του διαχωρισμού.

Η μέθοδος **partition()** εκτελεί το βήμα του διαχωρισμού. Ο πίνακας των πόλεων διαχωρίζεται με βάση την πυκνότητα των κρουσμάτων ανά 50000 κατοίκους, και η μέθοδος **quicksort()** εφαρμόζει αναδρομικά τον αλγόριθμο Quicksort στους υποπίνακες.

Κατά τον διαχωρισμό του πίνακα των πόλεων, η σύγκριση των πυκνοτήτων των κρουσμάτων ανά 50000 κατοίκους γίνεται χρησιμοποιώντας την μέθοδο **compareTo()** που υλοποιείται στην κλάση **City** σύμφωνα με τα ζητούμενα της εργασίας.

**Μέρος Β**

Στο Μέρος Β, υλοποιήσαμε, σύμφωνα με τις απαιτήσεις την ουρά προτεραιότητας **PQ**, η οποία περιλαμβάνει τις μεθόδους:

**int size()**: επιστρέφει τον αριθμό ενεργών στοιχείων στην ουρά. Έχει πολυπλοκότητα **O(1)** γιατί επιστρέφει την τιμή της μεταβλητής size που η τιμή της ενημερώνεται όποτε ένα στοιχείο εισάγεται ή αφαιρείται από την ουρά προτεραιότητας. Δεν χρειάζεται να επιτελεστεί επανάληψη σε ολόκληρη την ουρά για να μετρηθούν τα στοιχεία.

**boolean isEmpty()**: έλεγχος για το αν είναι άδεια η ουρά. Έχει πολυπλοκότητα **O(1)** γιατί χρησιμοποιεί την τιμή της μεταβλητής size για να ελέγξει αν η ουρά προτεραιότητας είναι άδεια. Εφόσον η μεταβλητή size ενημερώνεται αμέσως κατά την εισαγωγή ή αφαίρεση στοιχείων, η πρόσβαση σε αυτήν είναι άμεση και δεν εξαρτάται από το πλήθος των στοιχείων

**void insert(City x)**: εισαγωγή αντικειμένου. Έχει πολυπλοκότητα **O(log n)** γιατί το αντικείμενο προτίθεται άμεσα στο τέλος της ουράς (O(1)) και στην συνέχεια αναδύεται (swim). Κατά την ανάδυση, το νέο στοιχείο συγκρίνεται με τον πατέρα του και, αν είναι μικρότερο, ανταλλάσσεται με τον πατέρα του. Αυτή η διαδικασία επαναλαμβάνεται προς τα πάνω στο δέντρο, επειδή ο αριθμός των επιπέδων στο δέντρο είναι log(n), η διαδικασία swim έχει πολυπλοκότητα O(logn). Συνεπώς, η συνολική πολυπλοκότητα της λειτουργίας insert είναι O(logn). Αυτά ισχύουν σε περίπτωση που δεν χρειάζεται να γίνει resize, για την επέκταση του πίνακα.

**City min()**: επιστρέφει το στοιχείο με τη μικρότερη τιμή (μέγιστη προτεραιότητα) χωρίς να το αφαιρεί από την ουρά. Έχει πολυπλοκότητα **O(1)** γιατί απλώς επιστρέφει το στοιχείο με την ελάχιστη τιμή, το οποίο βρίσκεται στην ρίζα του σωρού.

**City getmin()**: Αφαιρεί και επιστρέφει το αντικείμενο με τη μικρότερη τιμή (μέγιστη προτεραιότητα). Έχει πολυπλοκότητα **O(log n)**, γιατί όταν αφαιρείται το αντικείμενο με τη μικρότερη τιμή που βρίσκεται στην ρίζα της σωρού, στην θέση του έρχεται το τελευταίο αντικείμενο το οποίο στην συνέχεια βυθίζεται για να πάει στην θέση του(sink). Αυτή η διαδικασία επηρεάζεται από τον αριθμό των επιπέδων στο δέντρο που είναι log(n).

**City max():** επιστρέφει το στοιχείο με τη μεγαλύτερη τιμή (ελάχιστη προτεραιότητα) χωρίς να το αφαιρεί από την ουρά. Έχει πολυπλοκότητα **O(n)** γιατί διατρέχει όλον τον σωρό για να βρει το μεγαλύτερο.

**City remove(int id)**: Αφαιρεί και επιστρέφει την πόλη με το δοθέν id. Μετά την αφαίρεση πρέπει να αποκατασταθεί η ιδιότητα του σωρού ώστε να μην καταστραφεί η ουρά.

Για να υλοποιήσουμε την remove() με πολυπλοκότητα **O(log n)** χρησιμοποιήσαμε μια επιπλέον δομή δεδομένων για να αποθηκεύσουμε τη θέση κάθε στοιχείου μέσα στον πίνακα **heap**. Αυτή η δομή δεδομένων είναι ένας πίνακας ακεραίων **indexMap[1000]** γιατί 1000 είναι τα διαφορετικά ID των πόλεων, όπου το indexMap[i] θα περιέχει τη θέση του στοιχείου με ID i στον πίνακα heap.

Έτσι χρησιμοποιούμε τον πίνακα indexMap[] για να εντοπίσουμε άμεσα τη θέση του στοιχείου που πρόκειται να αφαιρεθεί, και στη συνέχεια τοποθετούμε στην θέση του το

τελευταίο στοιχείο του ελαχιστοστρεφούς σωρού, έπειτα εκτελούμε την διαδικασία sink() για να διατηρήσουμε την ιδιότητα του ελαχιστοστρεφούς σωρού η οποία έχει πολυπλοκότητα O(log n).

**Μέρος Γ**

Η εκτέλεση του προγράμματος, γίνεται από την γραμμή εντολών με την μορφή:

**java DynamicInfluenza_k_withPQ <k> <input_file>**

όπου k το πλήθος των πόλεων με την μικρότερη πυκνότητα κρουσμάτων και input_file το όνομα του txt αρχείου. Όταν το αρχείο δεν είναι στο ίδιο φάκελο με το πρόγραμμα, τότε πρέπει να δώσουμε το path. Ως παράδειγμα, αν ο κώδικας είναι στο φάκελο src, και το αρχείο εισόδου έχει όνομα inputfile.txt και είναι στον φάκελο data, που βρίσκεται μέσα στον φάκελο src, τότε ως όνομα αρχείου πρέπει να δώσουμε data/inputfile.txt

Στο μέρος Γ, χρησιμοποιούμε έναν ελαχιστοστρεφή σωρό **priorityQueue** με μέγεθος 2k για να μην χρειαστεί να γίνει ποτέ η resize.

Στον σωρό αυτό τοποθετούμε μία προς μία τις πρώτες k πόλεις του αρχείου εισόδου και ταυτόχρονα κρατάμε στην μεταβλητή **maxOfMins** την τιμή της μεγαλύτερης πυκνότητας κρουσμάτων που υπολογίζουμε κάνοντας χρήση της μεθόδου cityDensityRatio() και στην μεταβλητή **IDmaxOfMins** το ID της πόλης αυτής.

Στην συνέχεια κάθε νέα πόλη εισέρχεται στον σωρό **μόνο** αν η πυκνότητα κρουσμάτων της πόλης αυτής **είναι μικρότερη από την μεγαλύτερη πυκνότητα πιθανότητας που υπάρχει ήδη στον σωρό** (maxOfMins), γιατί στην αντίθετη περίπτωση δεν υπάρχει λόγος να μπει στο σωρό. Πριν μπει στο σωρό η νέα πόλη, **αφαιρείται** από αυτόν η πόλη που είχε την προηγούμενη μεγαλύτερη πυκνότητα κρουσμάτων.

Κάθε πέντε γραμμές του αρχείου εμφανίζουμε τις k πόλεις με την μικρότερη πυκνότητα πιθανότητας με την μέθοδο **printTopCities()**. Η μέθοδος printTopCities() χρησιμοποιεί έναν προσωρινό ελαχιστοστρεφή σωρό **tempQueue** για να μην πειραχτούν τα στοιχεία του σωρού priorityQueue. Στην συνέχεια με την χρήση της μεθόδου getmin() αδειάζει ο σωρός tempQueue και εμφανίζονται σε αύξουσα σειρά τα περιεχόμενά του.

Σχετικά με την πολυπλοκότητα του Μέρους Γ:

Για την ανάγνωση του αρχείου έχουμε τόσες επαναλήψεις όσες και το πλήθος των εγγραφών του, άρα O(n).

Η εισαγωγή στον ελαχιστοστρεφή σωρό (insert) έχει πολυπλοκότητα O(log n).

Ο έλεγχος της πυκνότητας (cityDensityRatio και priorityQueueDensityRatio) καθώς και η εκτύπωση των k κορυφαίων πόλεων εκτελούνται σε σταθερό χρόνο, άρα O(1).

Συνολικά, η πολυπλοκότητα του προγράμματος εξαρτάται από τον αριθμό των πόλεων (n) στο αρχείο εισόδου. Η επανάληψη για κάθε γραμμή του αρχείου είναι O(n), και για κάθε επανάληψη, υπάρχει μια εισαγωγή στην ουρά προτεραιότητας, η οποία είναι O(log n). Επομένως, η συνολική πολυπλοκότητα αναμένεται να είναι O(n log n).

**<u>Μέρος Δ</u>**

Η εκτέλεση του προγράμματος, γίνεται από την γραμμή εντολών με την μορφή:

**java Dynamic_Media <input_file>**

όπου input_file το όνομα του txt αρχείου.

Χρησιμοποιούμε δύο ελαχιστοστρεφείς ουρές προτεραιότητας, την LeftHeap και την RightHeap.

Η ιδέα είναι στην LeftHeap να βάλουμε τις μισές πόλεις (αυτές που έχουν τους μικρότερους αριθμούς κρουσμάτων ανά 50000 κατοίκους) και στην RightHeap τις υπόλοιπες μισές (αυτές που έχουν τους μεγαλύτερους αριθμούς κρουσμάτων ανά 50000 κατοίκους), φροντίζοντας παράλληλα η LeftHeap να μην έχει περισσότερα στοιχεία από την RightHeap.

Με τον τρόπο αυτό όλες οι πόλεις της RightHeap, έχουν περισσότερα κρούσματα ανά 50000 κατοίκους από τις πόλεις της LeftHeap.

Στο τέλος αυτής της διαδικασίας αν η RightHeap έχει περισσότερα στοιχεία από την LeftHeap, τότε ο median είναι το μικρότερο στοιχείο της RightHeap. Αν οι δύο ουρές έχουν το ίδιο πλήθος στοιχείων ο median είναι ο μικρότερος από τον μικρότερο της RightHeap και τον μεγαλύτερο της LeftHeap.

Για να το πετύχουμε αυτό ακολουθούμε επαναληπτικά (μέχρι να τελειώσουν οι πόλεις) τα εξής βήματα:

1. Κάθε πόλη στοιχείο εισέρχεται (insert()) πρώτα στην RightHeap
2. Η πόλη με το μικρότερο αριθμό κρουσμάτων ανά 50000 κατοίκους της RightHeap εξάγεται από αυτήν (getmin()) και εισάγεται στην LeftHeap (insert())
3. Ελέγχουμε το πλήθος των πόλεων (size()) της LeftHeap και αν είναι περισσότερες από τις πόλεις της RightHeap, εξάγουμε (remove()) την πόλη **με το μεγαλύτερο αριθμό κρουσμάτων** ανά 50000 κατοίκους της LeftHeap και την εισάγουμε στην RightHeap (insert()).

Σχετικά με την πολυπλοκότητα του Μέρους Δ:

Για την ανάγνωση του αρχείου έχουμε τόσες επαναλήψεις όσες και το πλήθος των εγγραφών του, άρα O(n).

Η εισαγωγή στον ελαχιστοστρεφή σωρό (insert) έχει πολυπλοκότητα O(log n).

Ο έλεγχος της πυκνότητας (cityDensityRatio και priorityQueueDensityRatio) καθώς και η εκτύπωση των k κορυφαίων πόλεων εκτελούνται σε σταθερό χρόνο, άρα O(1).

Η εύρεση του μεγαλύτερου (max) έχει πολυπλοκότητα O(n).

Επομένως, η συνολική πολυπλοκότητα αναμένεται να είναι O(n log n).